Day 1B — The Transformer & Self-Attention

> 🎯 **Goal**: Understand the Transformer architecture and self-attention mechanism deeply enough to reason about LLM behavior.

> ⏱️ **Time**: 1.5h theory + 1h practice

> 📦 **Deliverable**: Hand-traced attention computation for a 3-token sequence, annotated Transformer block diagram

---

Table of Contents

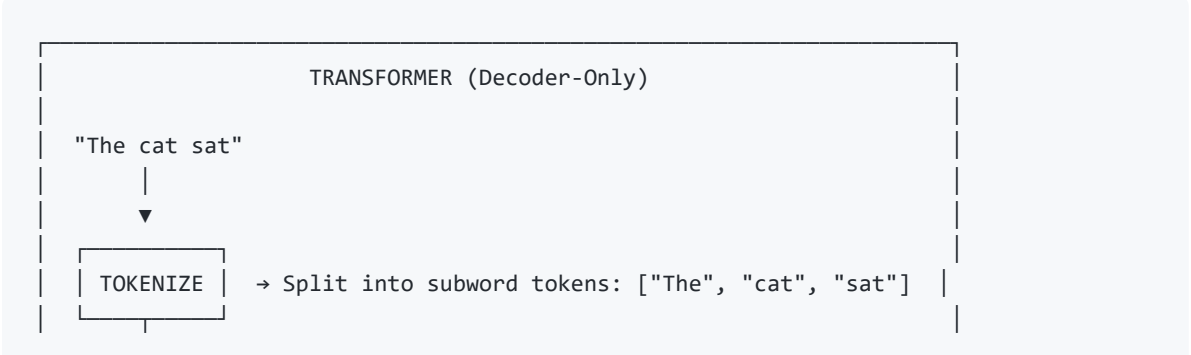
1. The Transformer Architecture (Overview)
2. Self-Attention — The Core Innovation
3. Multi-Head Attention
4. Inside the Transformer Block
5. Practice Exercises

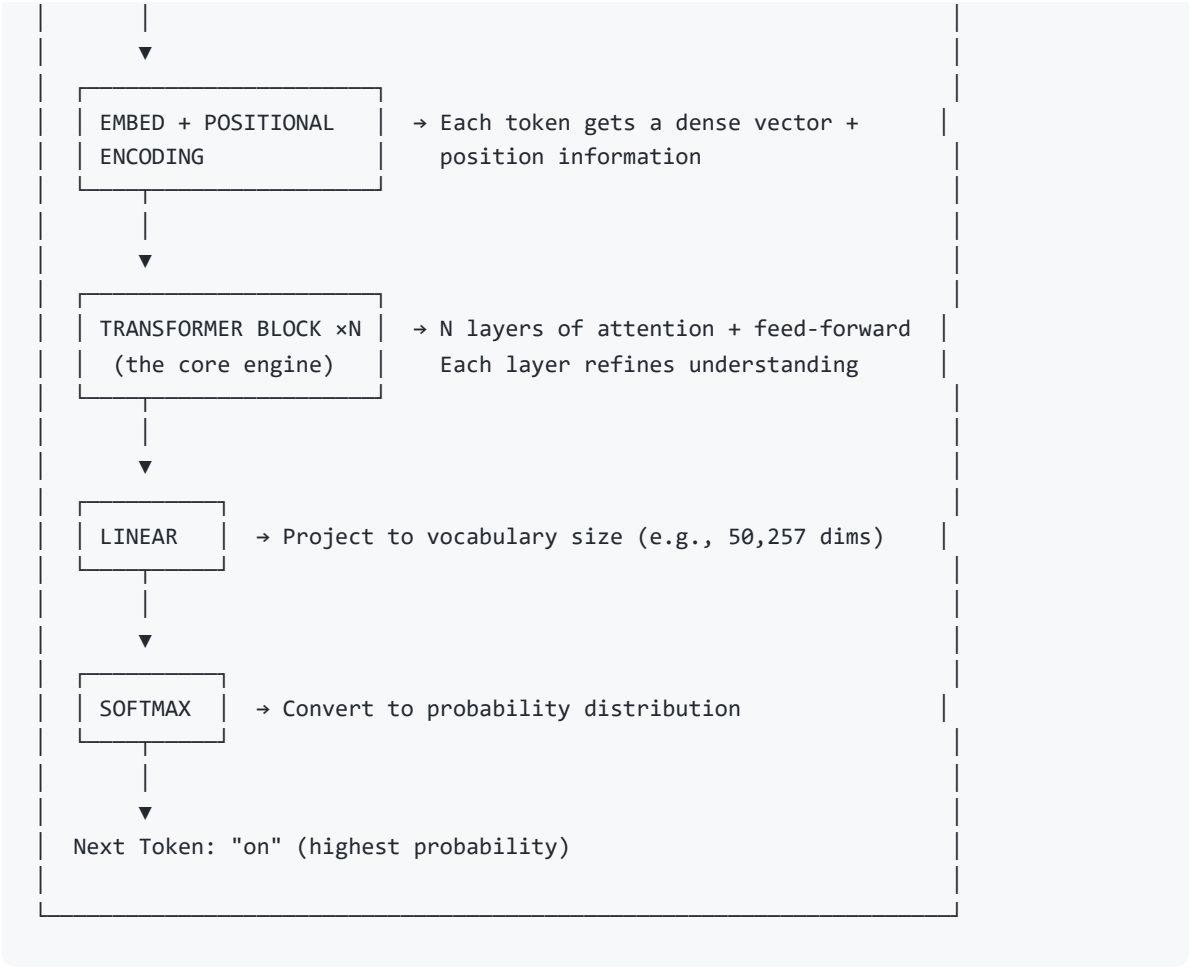
The Transformer Architecture (Overview)

Before 2017, sequence models meant RNNs and LSTMs — architectures that processed tokens **one at a time**, left to right. They were slow (no parallelism) and struggled with long-range dependencies (by the time you reach token 500, you've forgotten token 1).

The 2017 paper **"Attention Is All You Need"** (Vaswani et al.) changed everything. The Transformer processes **all tokens simultaneously** using attention, enabling massive parallelism on GPUs and capturing long-range relationships trivially.

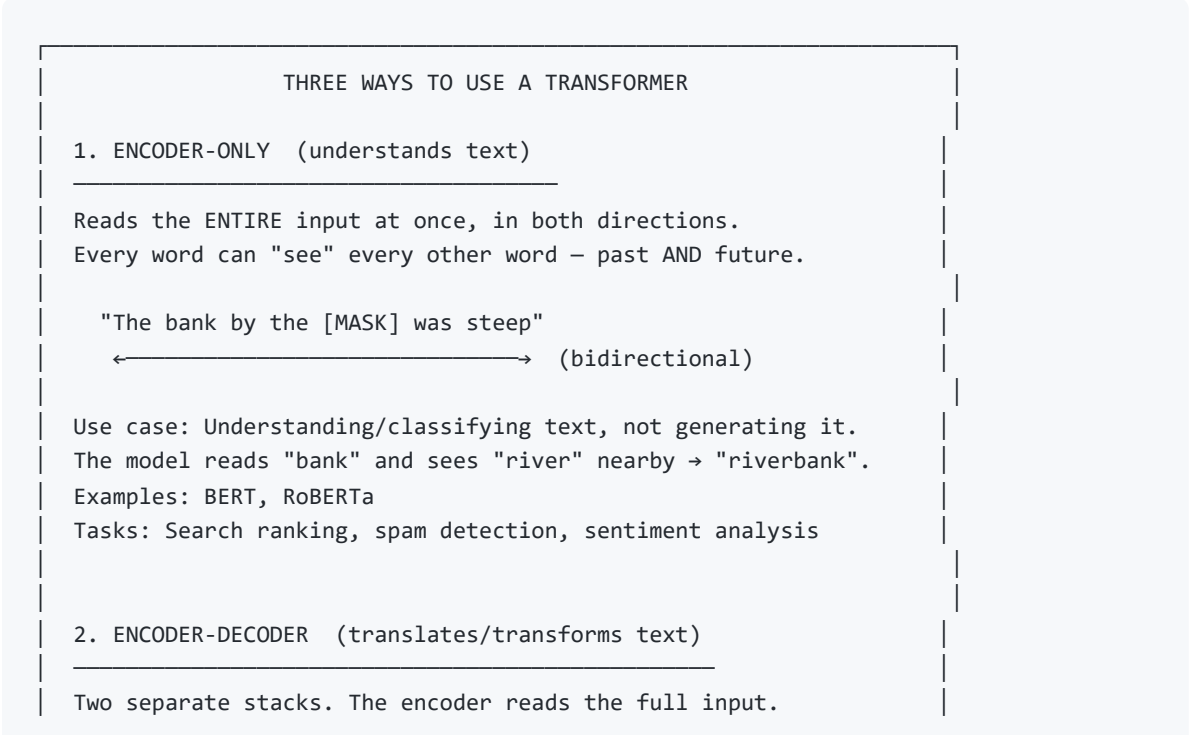
Big-Picture Pipeline





Three Transformer Variants

Not all Transformers are the same. The original paper had both an encoder and decoder, but the field has diverged into three families. Think of it like different reading strategies:



```
│  The decoder generates output one token at a time,                  │
│  while looking back at the encoder's understanding.                 │
│                                                                      │
│    Encoder: "The cat sat on the mat"  →  [understanding]            │
│                                                 ↓                   │
│    Decoder: "Le" → "chat" → "s'est" → "assis" → ...                 │
│                                                                      │
│  Use case: When input and output are different "languages."          │
│  Examples: T5, BART                                                 │
│  Tasks: Translation, summarization, format conversion               │
│                                                                      │
│                                                                      │
│  3. DECODER-ONLY  (generates text)  ← THIS IS WHAT ChatGPT IS       │
│  ──────────────────────────────────────────────────────────────     │
│  Reads left-to-right ONLY. Each token can only see tokens           │
│  that came BEFORE it. Predicts the next token, one at a time.       │
│                                                                      │
│    "The capital of France is" → ???                                 │
│     ←──────────────────────→    ↑                                   │
│     can see these tokens        predicts "Paris"                    │
│                                                                      │
│  Use case: Everything — chat, code, reasoning, translation.         │
│  It turns out next-token prediction is all you need at scale.       │
│  Examples: GPT-5, LLaMA 4, Claude Opus, Gemini 3                    │
│  Tasks: All of the above (via prompting)                            │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

Summary:

	Encoder-Only	Encoder-Decoder	Decoder-Only
Sees	All tokens (bidirectional)	All (enc), past only (dec)	Only past tokens (causal)
Generates?	No (classifies)	Yes (seq→seq)	Yes (open-ended)
Analogy	Reading an essay and grading it	Reading a book, then writing a book report	Writing a story one word at a time
Status (2026)	Niche	Some use cases	Dominant

Why Decoder-Only Won

Modern LLMs (GPT-5, Claude Opus 4.6, LLaMA 4, Gemini 3) are **all decoder-only**. Why?

1. **Simplicity** — One architecture, one training objective (predict next token)
2. **Scaling** — Decoder-only scales more predictably with compute
3. **Emergent abilities** — At sufficient scale, next-token prediction produces reasoning, coding, translation — all from one objective

4. **Flexibility** — Classification, generation, translation, coding — a single decoder-only model can do it all via prompting

The "N×" Stacking — Why Depth Matters

Each Transformer layer refines the model's internal representation. Early layers capture surface-level patterns (syntax, word associations). Later layers capture abstract reasoning and long-range dependencies.

```
Layer  1–10:  "This word is a noun, that word is a verb"
Layer 11–30:  "This sentence is about finance, not rivers"
Layer 31–60:  "The argument in paragraph 2 contradicts paragraph 5"
Layer 61–96:  "The user wants a Python function that sorts by date"

Model sizes:
  GPT-3:           96 layers,  175B parameters
  LLaMA-7B:        32 layers,    7B parameters
  LLaMA-70B:       80 layers,   70B parameters
  GPT-4:         ~120 layers,   ~1.8T parameters (rumored MoE)
  Claude Opus 4.6: undisclosed, estimated 500B+ parameters
  GPT-5.1/5.2:    undisclosed, likely MoE architecture
```

More layers = deeper understanding, but with diminishing returns and higher cost.

Self-Attention — The Core Innovation

The Intuition: Why We Need Attention

Consider the word "**bank**":

- "I sat by the river **bank**" → bank = edge of a river
- "I deposited money at the **bank**" → bank = financial institution

A simple word embedding gives "bank" **one fixed vector** regardless of context. But we need **different representations** depending on surrounding words. How does the model decide which meaning to use?

It looks at the other tokens in the sentence. That's attention.

```
"I deposited money at the bank"

Without attention:  "bank" = [0.5, 0.3, -0.2, 0.8]  (always the same)

With attention:
  "bank" sees "money", "deposited" → activates financial meaning
```

```
  "bank" = [0.1, 0.9, -0.4, 0.2]  (context-aware: financial)

"I sat by the river bank"

With attention:
  "bank" sees "river", "sat" → activates geographical meaning
  "bank" = [0.8, 0.1, 0.6, -0.3]  (context-aware: riverside)
```

Attention creates **context-dependent embeddings** — every token's representation is shaped by the tokens around it.

The Q, K, V Framework — A Library Analogy

This is the most important concept in this guide. Let's build the intuition carefully.

Imagine you walk into a library looking for information about a topic.

```
┌─────────────────────────────────────────────────────────────┐
│                    THE LIBRARY ANALOGY                      │
│                                                             │
│  YOU (a token) walk into the library with a QUESTION:       │
│                                                             │
│    Query (Q): "I need information about rivers"             │
│                                                             │
│  Each book on the shelf has a LABEL on its spine:           │
│                                                             │
│    Key (K):  "Geography"  "Finance"  "Rivers of Europe"     │
│                                                             │
│  You compare your question to each label:                   │
│                                                             │
│    Q · K₁ = "rivers" vs "Geography"       → moderate match  │
│    Q · K₂ = "rivers" vs "Finance"         → low match       │
│    Q · K₃ = "rivers" vs "Rivers of Europe"→ high match!     │
│                                                             │
│  Then you READ the content of the most relevant books:      │
│                                                             │
│    Value (V): The actual detailed content inside each book  │
│                                                             │
│  Your output = weighted combination of book contents,       │
│  weighted by how relevant each book's label was to your query │
└─────────────────────────────────────────────────────────────┘
```

Every token in the sequence plays ALL three roles simultaneously:

```
Token "bank" is:
  - A VISITOR asking questions about other tokens  (Query)
  - A BOOK being searched by other tokens          (Key)
  - A CONTENT PROVIDER giving info to other tokens (Value)
```

```
All tokens query all other tokens at the same time.
This is what makes it "self"-attention — the sequence attends to itself.
```

Why Three Separate Projections (Q, K, V)?

You might wonder: **Why not just compare the raw embeddings directly?**

Because the **question you ask**, the **label you present**, and the **content you provide** are fundamentally different things:

```
Consider the token "bank":

  As a Query (what it's LOOKING FOR):
    "I need context — am I financial or geographical?"

  As a Key (what it ADVERTISES to others):
    "I'm a noun that could be a place or institution"

  As a Value (what it CONTRIBUTES when attended to):
    "Here's my detailed semantic content about institutions/places"

These are three different roles! A single vector can't optimally
serve all three purposes. Learned projection matrices (W_Q, W_K, W_V)
let the model create SPECIALIZED representations for each role.
```

Step-by-Step Numerical Walkthrough

Let's trace attention for **3 tokens: "The", "cat", "sat"** using tiny 4-dimensional vectors.

Step 1: Start with embeddings

```
After embedding + positional encoding, each token has a vector:

  "The" = [1.0,  0.5, -0.5,  0.2]
  "cat" = [0.3,  0.8,  0.6, -0.1]
  "sat" = [0.7, -0.2,  0.4,  0.9]
```

Step 2: Project to Q, K, V using learned weight matrices

In practice, W_Q, W_K, W_V are learned during training. Here we use simple matrices for illustration (projecting 4-dim → 3-dim for clarity):

```
         ┌              ┐
         │ 1   0   1    │
W_Q  =   │ 0   1   0    │     (4×3 matrix)
         │ 1   0  -1    │
         │ 0   1   1    │
         └              ┘
```

```
          └              ┘

Q_the = [1.0, 0.5, -0.5, 0.2] × W_Q = [0.5,  1.2,  1.2]
Q_cat = [0.3, 0.8,  0.6, -0.1] × W_Q = [0.9,  0.7, -0.1]
Q_sat = [0.7, -0.2, 0.4,  0.9] × W_Q = [1.1,  0.7,  0.2]

Similarly for K and V (using different W_K, W_V matrices):

K_the = [0.8,  0.4,  0.6]      V_the = [0.3,  0.7,  0.5]
K_cat = [0.5,  0.9,  0.3]      V_cat = [0.8,  0.2,  0.6]
K_sat = [0.6,  0.1,  0.8]      V_sat = [0.4,  0.9,  0.3]
```

Step 3: Compute attention scores (Q × Kᵀ)

Each query asks: "How relevant is each key to me?"

```
Score = Q × Kᵀ  (matrix multiplication)

                    K_the   K_cat   K_sat
                    [0.8]   [0.5]   [0.6]
                    [0.4]   [0.9]   [0.1]
                    [0.6]   [0.3]   [0.8]

Q_the [0.5, 1.2, 1.2]  →  [1.60,   1.69,   1.26]
Q_cat [0.9, 0.7, -0.1]  →  [1.00,   1.05,   0.53]
Q_sat [1.1, 0.7, 0.2]   →  [1.28,   1.22,   0.89]

Score Matrix:
              The    cat    sat
    The  [   1.60   1.69   1.26  ]
    cat  [   1.00   1.05   0.53  ]
    sat  [   1.28   1.22   0.89  ]
```

Step 4: Scale by √d_k

```
WHY SCALE? Without scaling, when d_k is large (e.g., 64), the dot
products become very large. Large values pushed into softmax produce
extremely peaked distributions (one value ≈ 1.0, rest ≈ 0.0).
This kills the gradient — the model can't learn!

Scaling by √d_k keeps values in a moderate range where softmax
produces useful (non-saturated) gradients.

d_k = 3 (our key dimension), so √d_k = √3 ≈ 1.73

Scaled scores = Score / 1.73:

              The    cat    sat
    The  [   0.92   0.98   0.73  ]
    cat  [   0.58   0.61   0.31  ]
    sat  [   0.74   0.71   0.51  ]
```

Step 5: Apply softmax (row-wise)

```
WHY SOFTMAX? We need scores to become probabilities (positive, sum to 1).
Softmax is differentiable (critical for backpropagation) and amplifies
differences — higher scores get disproportionately more weight.

softmax(x_i) = e^(x_i) / Σ(e^(x_j))

Row "The": e^0.92=2.51, e^0.98=2.66, e^0.73=2.08  → sum=7.25
  → [2.51/7.25, 2.66/7.25, 2.08/7.25] = [0.346, 0.367, 0.287]

Row "cat": e^0.58=1.79, e^0.61=1.84, e^0.31=1.36  → sum=4.99
  → [1.79/4.99, 1.84/4.99, 1.36/4.99] = [0.359, 0.369, 0.272]

Row "sat": e^0.74=2.10, e^0.71=2.03, e^0.51=1.67  → sum=5.80
  → [2.10/5.80, 2.03/5.80, 1.67/5.80] = [0.362, 0.350, 0.288]

Attention Weights:
              The    cat    sat
    The  [  0.346  0.367  0.287  ]   ← "The" attends most to "cat"
    cat  [  0.359  0.369  0.272  ]   ← "cat" attends most to itself
    sat  [  0.362  0.350  0.288  ]   ← "sat" attends most to "The"
```

Step 6: Multiply attention weights by V

```
Output = Attention_Weights × V

                     V_the    V_cat    V_sat
                    [0.3]    [0.8]    [0.4]
                    [0.7]    [0.2]    [0.9]
                    [0.5]    [0.6]    [0.3]

Out_the = 0.346×[0.3,0.7,0.5] + 0.367×[0.8,0.2,0.6] + 0.287×[0.4,0.9,0.3]
        = [0.104,0.242,0.173] + [0.294,0.073,0.220] + [0.115,0.258,0.086]
        = [0.513, 0.573, 0.479]

Out_cat = 0.359×[0.3,0.7,0.5] + 0.369×[0.8,0.2,0.6] + 0.272×[0.4,0.9,0.3]
        = [0.108,0.251,0.180] + [0.295,0.074,0.221] + [0.109,0.245,0.082]
        = [0.512, 0.570, 0.483]

Out_sat = 0.362×[0.3,0.7,0.5] + 0.350×[0.8,0.2,0.6] + 0.288×[0.4,0.9,0.3]
        = [0.109,0.253,0.181] + [0.280,0.070,0.210] + [0.115,0.259,0.086]
        = [0.504, 0.582, 0.477]
```

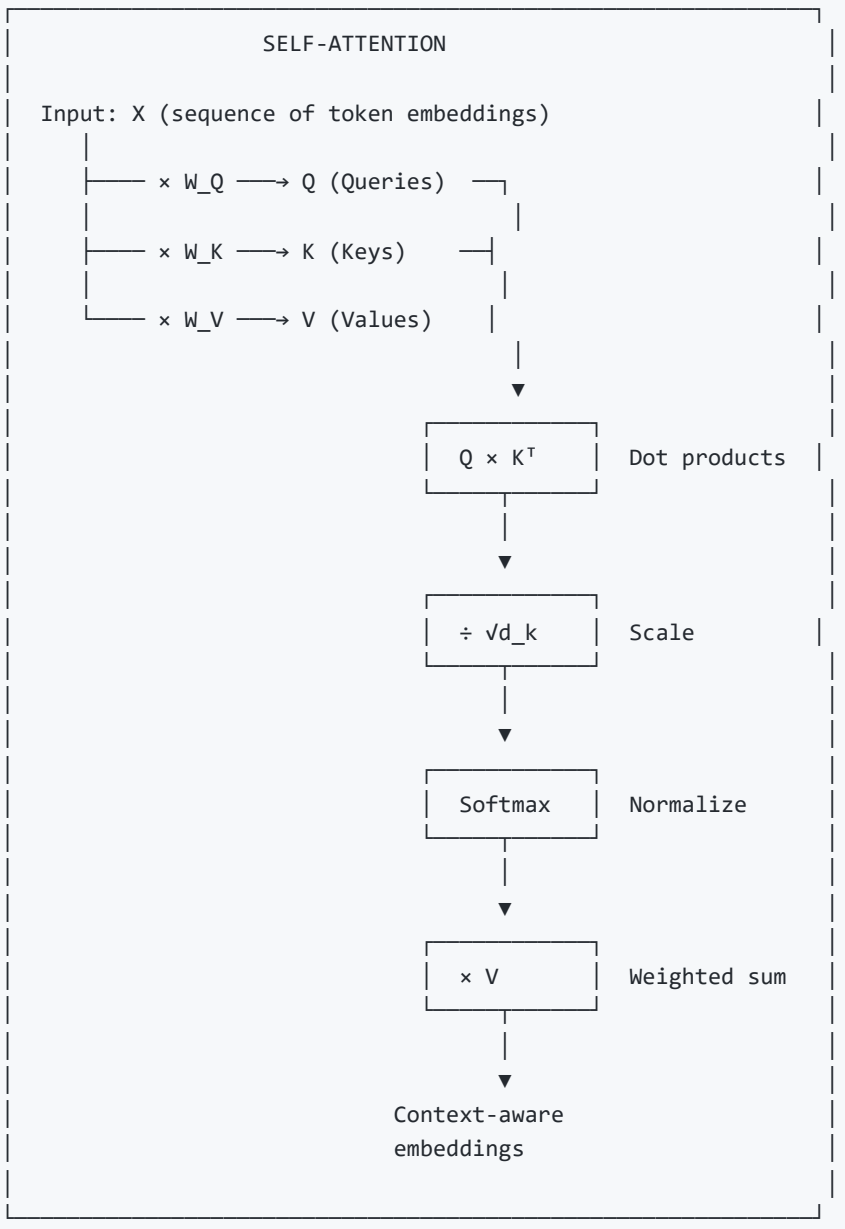
Step 7: What changed?

```
Before attention (raw embeddings — context-blind):
  "The" = [1.0,  0.5, -0.5,  0.2]
  "cat" = [0.3,  0.8,  0.6, -0.1]
  "sat" = [0.7, -0.2,  0.4,  0.9]
```

```
After attention (context-aware representations):
  "The" = [0.513, 0.573, 0.479]
  "cat" = [0.512, 0.570, 0.483]
  "sat" = [0.504, 0.582, 0.477]

Each token's output is now a BLEND of information from all tokens,
weighted by relevance. "The" now carries information about "cat"
and "sat". No token is an island — every representation is
shaped by its context.
```

Attention Computation Flow



Formula: $\text{Attention}(Q, K, V) = \text{softmax}(Q \times K^T / \sqrt{d_k}) \times V$

Multi-Head Attention

Why Multiple Heads?

A single attention head learns **one type of relationship** between tokens. But language has **many simultaneous relationships**:

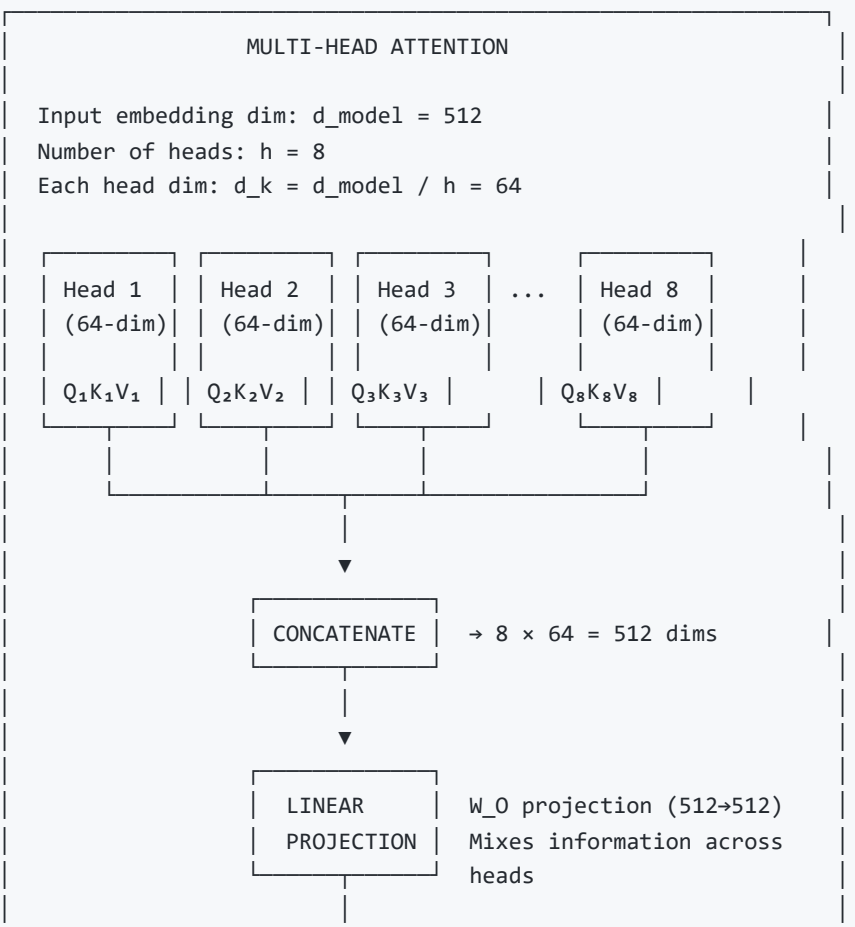
```
"Alice told Bob that she would meet him at the bank near the river"

Relationships a model needs to track simultaneously:

  Head 1 (Syntax):     "Alice" ↔ "told"  (subject-verb)
  Head 2 (Coreference): "she" → "Alice"    (pronoun resolution)
  Head 3 (Coreference): "him" → "Bob"      (pronoun resolution)
  Head 4 (Semantics):   "bank" ← "river"   (word sense: riverside)
  Head 5 (Position):    "at" ← "bank"      (prepositional attachment)
  Head 6 (Discourse):   "told" ← "meet"    (reported speech link)

One attention head can't capture all of these at once.
Multiple heads let the model attend to different relationship
types in parallel.
```

How Multi-Head Attention Works



```
│                          ▼                                    │
│                  Output (512-dim)                              │
│                                                               │
└──────────────────────────────────────────────────────────────┘
```

Typical Head Counts

```
Model          d_model    Heads    d_k (per head)
─────────────  ─────────  ──────   ──────────────
BERT-base        768        12          64
BERT-large      1024        16          64
GPT-2            768        12          64
LLaMA-7B        4096        32         128
LLaMA-70B       8192        64         128
GPT-3           12288        96         128
GPT-5.x        varies     GQA      varies (MoE)
```

Masked (Causal) Attention

For **text generation**, a critical constraint: when predicting the next token, the model **cannot look at future tokens**. You can't use the answer to predict the answer!

This is enforced with a **causal mask** — a triangular matrix that blocks attention to future positions:

```
Tokens:    "The"  "cat"  "sat"  "on"

Mask Matrix (1 = can attend, 0 = blocked):

            The   cat   sat    on
    The  [   1     0     0     0  ]   ← "The" can only see itself
    cat  [   1     1     0     0  ]   ← "cat" sees "The" + itself
    sat  [   1     1     1     0  ]   ← "sat" sees "The", "cat", itself
    on   [   1     1     1     1  ]   ← "on" sees everything before it

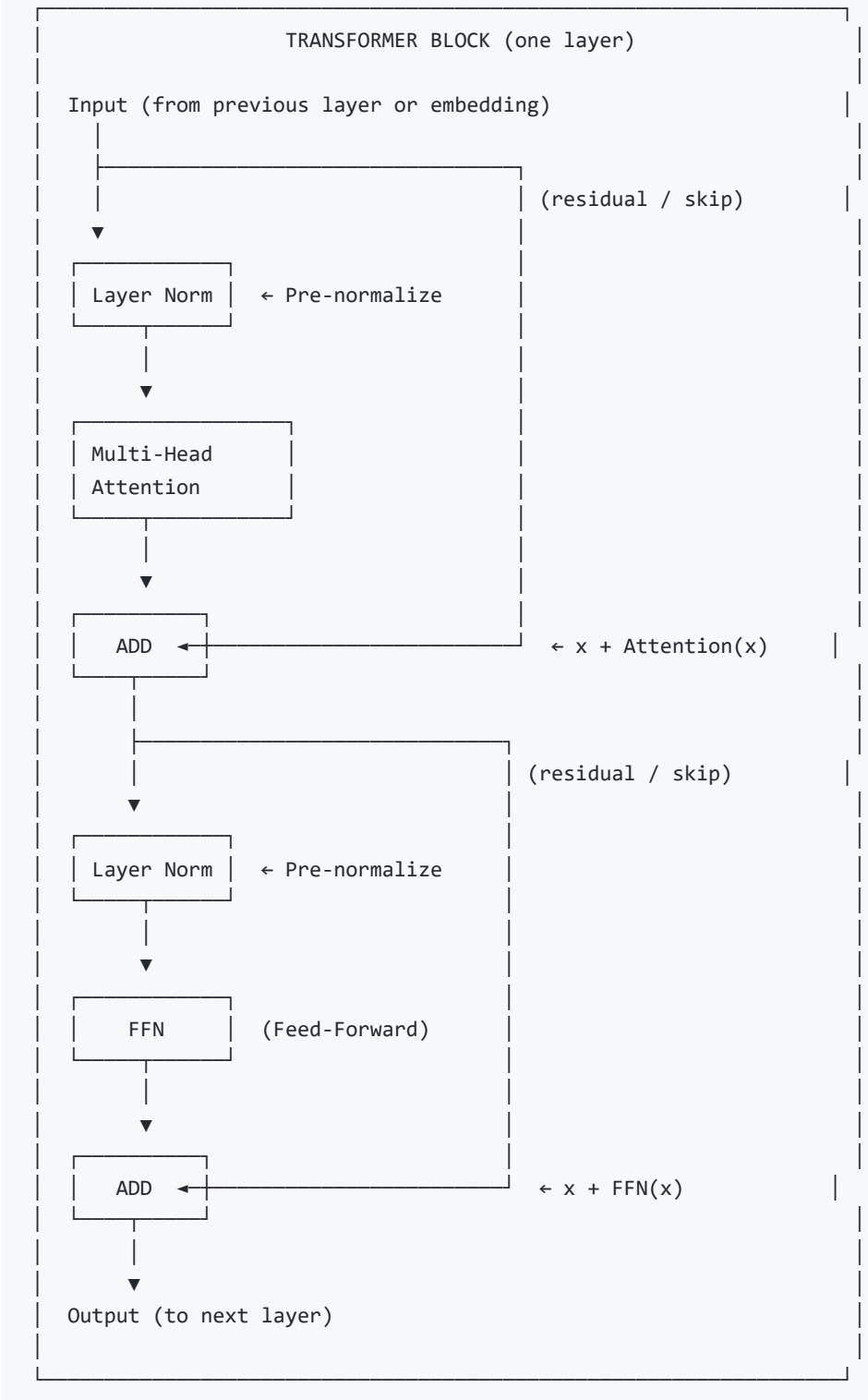
Implementation: set masked positions to -∞ BEFORE softmax.
softmax(-∞) = 0, so future tokens get zero attention weight.

This is why it's called "causal" — each token can only be
caused by (depend on) past tokens, never future ones.
This is what makes decoder-only models autoregressive.
```

Inside the Transformer Block

Each of the N transformer layers is a **block** with the same internal structure. Understanding this block is understanding the entire model.

Full Transformer Block



Layer Normalization

Why do we need it? As data flows through 32–96+ layers, values can explode (grow huge) or vanish (shrink to zero). Without normalization, deep networks become untrainable.

```
Problem without LayerNorm:

  Layer 1 output:  [0.5,  0.3, -0.2]        ← reasonable
  Layer 10 output: [15.2, -8.7,  22.1]      ← growing
  Layer 50 output: [4521, -2893, 7104]       ← exploded!

With LayerNorm after each sub-layer:

  Layer 1 output:  [0.5,  0.3, -0.2]  → normalize → [0.87, 0.22, -1.09]
  Layer 10 output: [0.8, -0.4,  0.6]  → normalize → [1.07, -1.34,  0.27]
  Layer 50 output: [0.3,  0.7, -0.1]  → normalize → [-0.51, 1.38, -0.87]
                                          ↑ stays bounded!
```

What it does:

1. Compute mean and standard deviation of the vector
2. Normalize: (x - mean) / std → mean=0, std=1
3. Scale and shift with **learned** parameters γ and β

```
LayerNorm(x) = γ × (x - μ) / σ + β

μ = mean of x,  σ = std of x
γ, β = learned parameters (the model decides the best scale/shift)
```

Pre-norm vs Post-norm:

```
Original paper (2017) — Post-norm:
  x → Attention → Add → LayerNorm → FFN → Add → LayerNorm

Modern models (GPT-5, LLaMA 4, Claude) — Pre-norm:
  x → LayerNorm → Attention → Add → LayerNorm → FFN → Add

Pre-norm trains more stably. The residual path stays "clean"
(un-normalized), which helps gradients flow through deep networks.
```

Feed-Forward Network (FFN) — The Knowledge Store

Why do we need FFN when we already have attention?

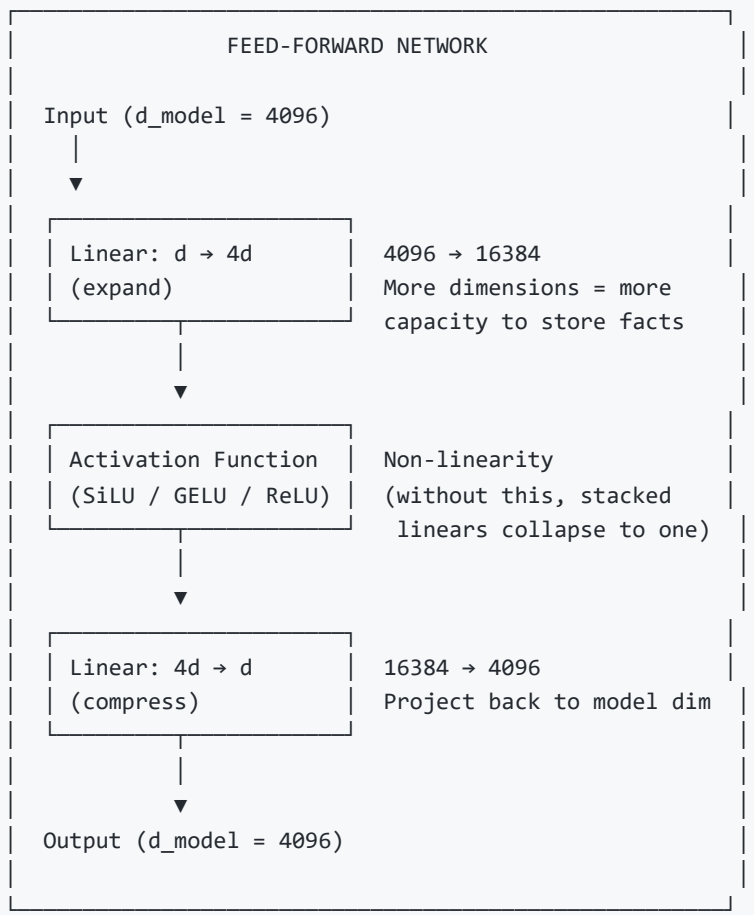
Attention decides **WHAT information to gather** from context. FFN decides **WHAT TO DO with that information**. They have complementary roles:

```
Analogy:
  Attention = Reading the right books in the library
  FFN       = Thinking about what you read and forming conclusions

Without FFN: The model can look at relevant context but can't
```

```
             reason about it or apply stored knowledge.
Without Attention: The model can think but has no way to
                   gather relevant context.
```

FFN Structure:



Why the 4× expansion? More parameters in the hidden layer = more capacity to **store factual knowledge**. Research shows that specific neurons in the FFN activate for specific facts:

```
Neuron 3847 in layer 22:  activates for "Paris is the capital of France"
Neuron 1204 in layer 35:  activates for Python syntax patterns
Neuron 8891 in layer 48:  activates for date formatting conventions

The FFN layers collectively form a massive key-value memory store.
The 4× expansion gives the model enough "slots" to store millions
of learned associations.
```

Activation functions — brief comparison:

```
┌──────────┬──────────────────────────┬──────────────────────┐
│ Function │ Formula                  │ Used In              │
├──────────┼──────────────────────────┼──────────────────────┤
```

ReLU	max(0, x)	Original Transformer
GELU	x × Φ(x) (Gaussian CDF)	BERT, GPT-2, GPT-3
SiLU	x × σ(x) (x × sigmoid)	LLaMA, Mistral, GPT-5, Claude
Trend	ReLU → GELU → SiLU (progressively smoother)	Smoother = better gradient flow

Residual (Skip) Connections

The `+` in `output = layer(input) + input` is the **residual connection** — and it's what makes deep Transformers possible.

Why are residual connections essential?

Without them, a 96-layer network would be untrainable. Gradients (the learning signal) must flow backwards through every layer during training. In deep networks without residuals, gradients **vanish** — they shrink exponentially until early layers receive essentially zero learning signal.

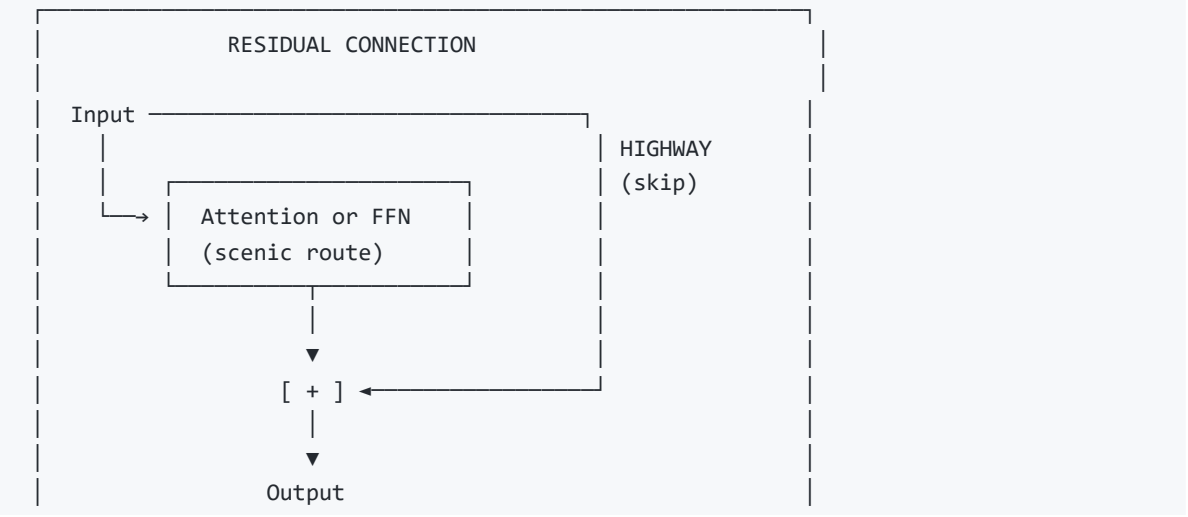
```
Without residual connections (96 layers):

  Gradient at layer 96: 1.0
  Gradient at layer 50: 0.0001
  Gradient at layer  1: 0.0000000000001  ← effectively zero, can't learn

With residual connections:

  Gradient at layer 96: 1.0
  Gradient at layer 50: 0.95    ← gradient flows through skip connection
  Gradient at layer  1: 0.82   ← still strong, can learn!
```

The highway analogy:



```
┌                                                          ┐
│  output = layer(input) + input                           │
│                                                          │
│  Each layer only needs to learn the DELTA (improvement)  │
│  not the entire representation from scratch.             │
│                                                          │
│  If a layer has nothing useful to add, it can learn      │
│  to output ≈ 0, and the input passes through unchanged.  │
│                                                          │
└                                                          ┘
```

This is why Transformers can be stacked so deep — each layer learns a **small refinement** on top of what's already there, rather than rebuilding the representation from scratch.

Practice Exercises

Exercise 1: Trace Attention by Hand

Take the sentence "**I love cats**" with these embeddings (3-dim for simplicity):

```
"I"    = [1.0, 0.0, 0.5]
"love" = [0.2, 1.0, 0.3]
"cats" = [0.8, 0.5, 1.0]
```

Using these weight matrices:

```
        ⌈         ⌉          ⌈         ⌉          ⌈         ⌉
W_Q =   | 1  0    |  W_K =   | 0  1    |  W_V =   | 1  1    |
        | 0  1    |          | 1  0    |          | 0  1    |
        | 1  1    |          | 1  1    |          | 1  0    |
        ⌊         ⌋          ⌊         ⌋          ⌊         ⌋
```

1. Compute Q, K, V for each token
2. Compute the score matrix $Q \times K^T$
3. Scale by $\sqrt{d_k}$ ($d_k = 2$)
4. Apply softmax row-wise
5. Compute the final output

Exercise 2: Draw the Causal Mask

For a 5-token sequence `["The", "dog", "runs", "very", "fast"]`:

1. Draw the 5×5 causal mask matrix

2. Show what the attention score matrix looks like **before** and **after** applying the mask
3. Explain why position 3 ("very") cannot attend to position 5 ("fast")

Exercise 3: Annotate a Transformer Block

Draw (on paper or digitally) a complete Transformer block diagram and annotate each component with:

1. Its purpose (one sentence)
2. Its input/output dimensions for LLaMA-7B (d_model=4096, heads=32, d_ff=11008)
3. Why it's necessary (what breaks if you remove it?)

✓ Day 1B Checklist

- ☐ Can explain what self-attention does and why it's needed (context-dependent embeddings)
- ☐ Can trace Q, K, V computation through a small example by hand
- ☐ Understand why Q, K, V are separate projections (different roles)
- ☐ Know why we scale by √d_k (prevent softmax saturation)
- ☐ Can explain multi-head attention (parallel relationship detection)
- ☐ Understand the causal mask and why it's needed for generation
- ☐ Can explain all components of a Transformer block (LayerNorm, FFN, residuals)
- ☐ Know the role of FFN vs attention (knowledge store vs context gathering)
- ☐ Can explain why residual connections enable deep networks

▶ 📝 **Answer Key (click to expand)**

Next → Day 1C: Training Pipeline & Inference